UVSQ M2 Datascale

Génération de Graphes RDF à partir de texte

Réalisé par : Mecherak Thanina · Mouhoun Lydia · Fellati Zineb · Tahiri Mouad

Encadré par : Pr. Kedad Zoubida

Objectif principal

- Transformer un texte brut en graphe de connaissances
- Extraire automatiquement les entités et les relations
- Représenter ces connaissances sous forme de graphe structuré

Plan de la présentation

01 État de l'art : 4 méthodes

Analyse des approches existantes

03 Solution proposée (Pipeline)

Architecture hybride NLP + LLM + RDF

02 Etude comparative

Comparaison des approches existantes

04 Implémentation technique

Stack, algorithmes, scores de confiance

05 Limites & perspectives

Points d'amélioration et travaux futurs

01

État de l'art

Etude de 4 méthodes de construction de graphes de connaissances

Vue d'ensemble : 4 méthodes étudiées

[A1]

Debatty et al., 2014

Graphe de similarité

MapReduce + CTPH
k-NN Graph

[A2]

Atzori, 2022

Knowledge Graph léger

Embeddings non supervisés
KGQA + BESt Queries

[A3]

Novelli & Oliveira, 2011

Ontologie OWL

LSA + SVD + Clustering
WordNet + OWL

[A4]

Agrawal et al., 2023

KG Cybersécurité

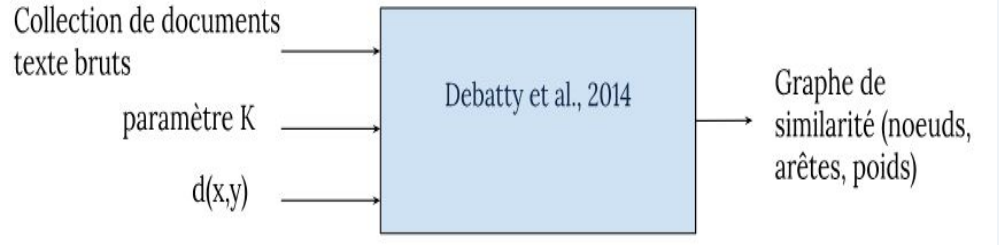
NLP + Expert humain
Neo4j + spaCy

Toutes les approches : Bottom-up · Non structuré → Structuré · Texte brut en entrée

Article 1: Algorithme NNCTPH et graphe de similarité syntaxique

[A1] Debatty et al., 2014— Scalable Graph Building

- Objectif : Graphe k-NN pondéré à grande échelle
- CTPH (signature du document) + LSH (partitionnement et regroupement)
- Distance Jaro-Winkler/Levenshtein pour précision
- MapReduce / distribué
- Input => **corpus, paramètre k, d(x, y)**
- Output => Le graphe de similarité (**nœuds, arêtes, poids**)
- Similarité syntaxique, pas sémantique



Article 2: Extracting Data from Text and Querying it via KGQA and BESt Queries

[A2] Atzori, 2022

Objectif: Construire automatiquement un Knowledge Graph à partir de textes bruts, sans ontologie ni annotations manuelles.

Méthodologie : OKgraph utilise des word embeddings pour :

- Set Expansion : trouver des entités similaires
- Relation Expansion : identifier des relations entre entités

Exploitation du graphe

- Questions en langage naturel (KGQA)
- Requêtes structurées par exemple (BESt Queries)

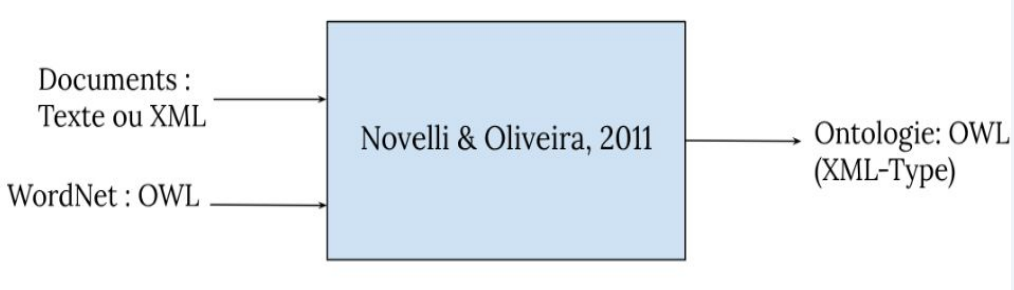
Limitations

- Ontologie peu formelle
- Pas de validation globale du graphe

Article 3: Simple Method for Ontology Automatic Extraction from Documents

[A3] Novelli & Oliveira, 2011— Extraction d'ontologies OWL

- Objectif : Ontologie OWL depuis corpus
- TF-IDF → termes index
- LSA (SVD) → réduction dimensionnelle
- K-Means / agglomératif → hiérarchie
- WordNet → relations sémantiques
- Totalement automatisé, agnostique domaine
- Scalabilité limitée sur grands corpus



Article 4: Ontologie automatique et KG en cybersécurité

[A4] Agrawal et al., 2023— KG pédagogique cybersécurité

- **Objectif** : Construire un graphe de connaissances en cybersécurité à partir de textes non structurés.

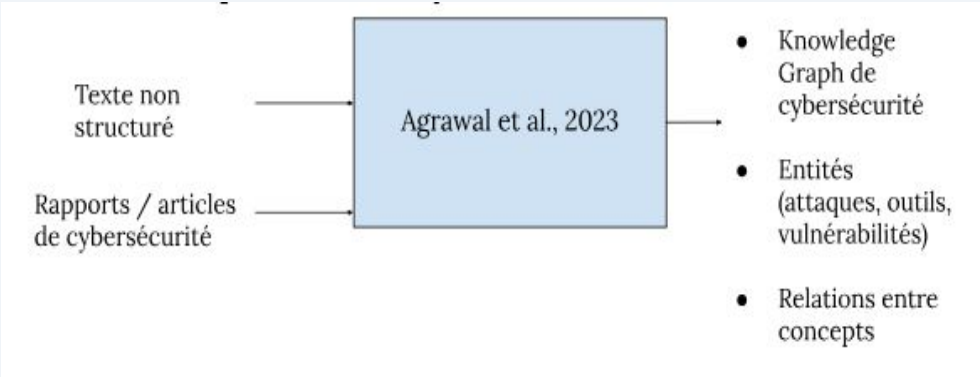
- **Approche:**
  - NLP + POS tagging pour extraire entités et relations
  - Création de triplets (entité–relation–entité)
  - Validation et structuration par un expert

Architecture en 3 phases :

- **1** Acquisition des connaissances
  Extraction NLP + développement de l'ontologie
- **2** Stockage
  Base de graphe Neo4j + validation avec ReGraph
- **3** Utilisation
  interface Web + chatbot basé sur SVM

- **Limite :**

  Forte dépendance à l'expertise humaine



02

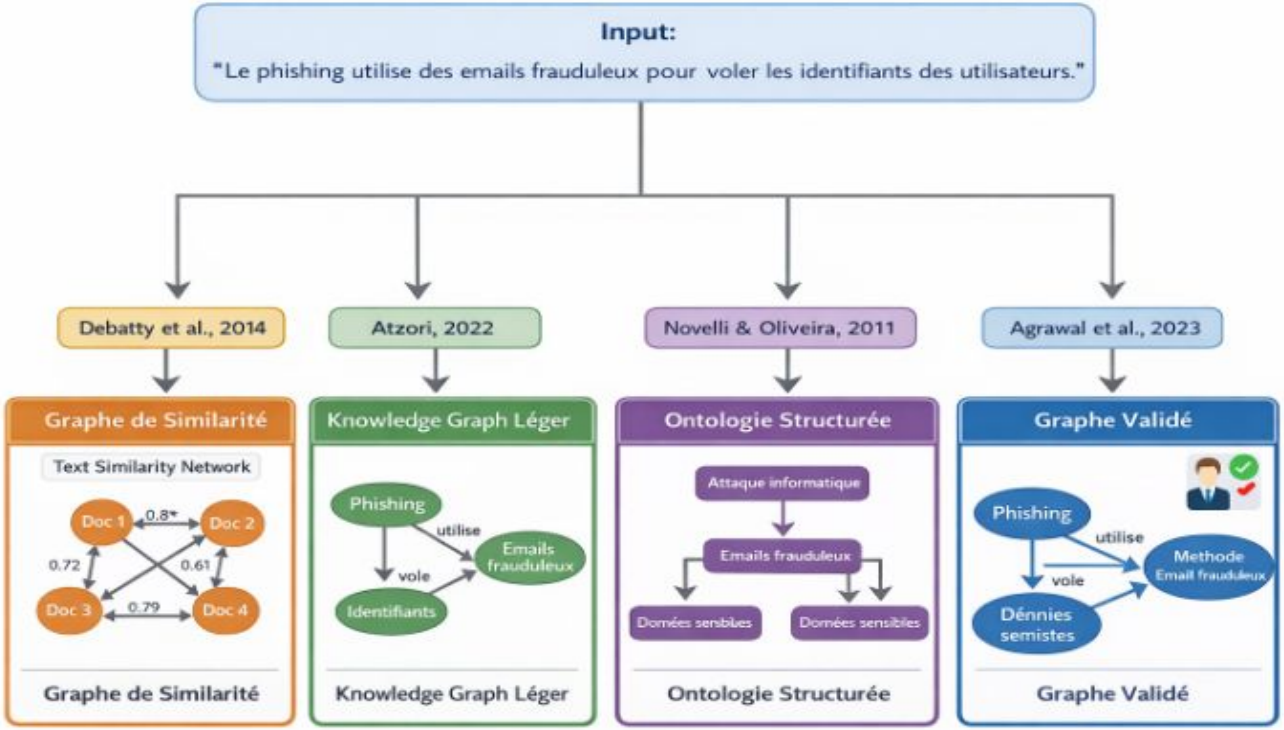
Étude comparative

Comparaison des approches existantes

Comparaison synthétique des 4 méthodes

Critère	[A1] Debatty et al., 2014	[A2] Atzori, 2022	[A3] Novelli & Oliveira, 2011	[A4] Agrawal et al., 2023
Sortie	Graphe k-NN	Knowledge Graph	Ontologie OWL	KG pédagogique
Approche	Hachage + LSH + Distance	Embeddings NLP	LSA + Clustering	NLP + Humain
Automatisation	Élevée	Élevée	Totale	Semi-auto
Ontologie	Aucune	Légère	OWL générée	Ontologie définie par experts
Sémantique	Syntaxique seul	Proximité vectorielle	WordNet	Règles + expert
Validation de la cohérence	Comparaison avec la réalité terrain	Non explicitement traitée	Validation d'un expert	Cas d'usage métier

Sorties de chaque approche

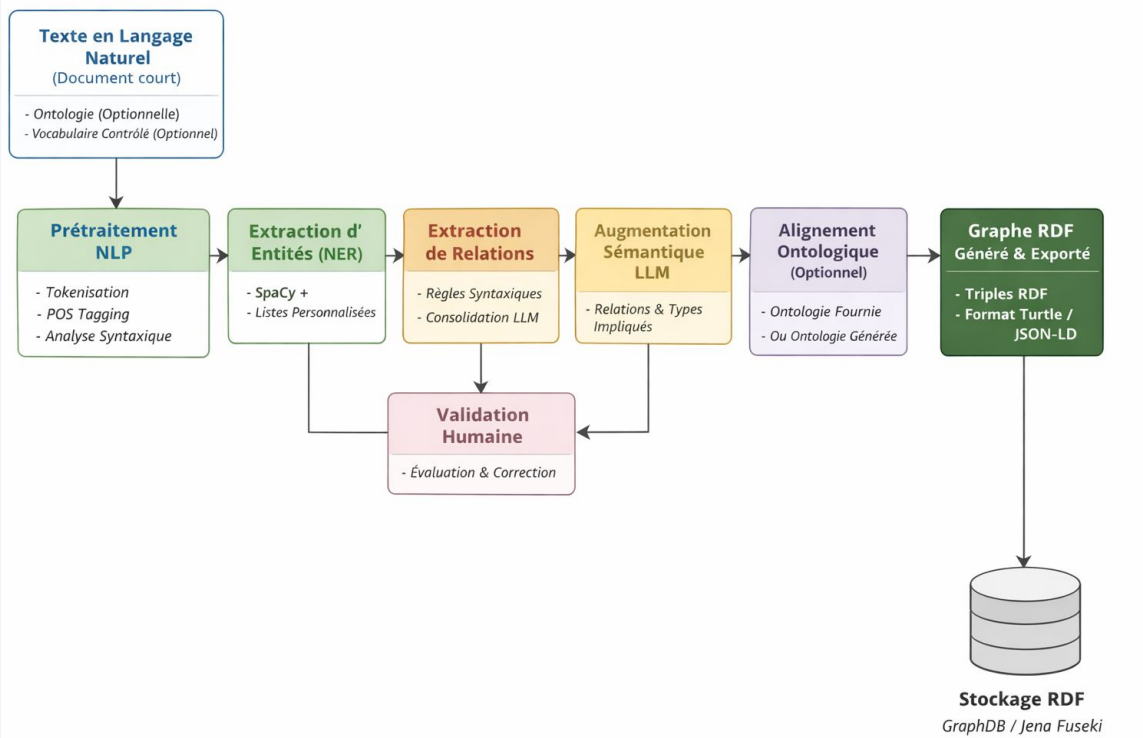


03

Solution Proposée

Pipeline hybride : NLP symbolique · LLM neuronal · RDF · Human-in-the-loop

Vue d'ensemble du Pipeline: Boîte à Outils



Étapes 3 & 4: NER & Extraction des Relations

NER : Extraction des Entités

- NER avec SpaCy pour identifier les entités (PERSON, ORG, PRODUCT, etc.)
- Vocabulaire contrôlé optionnel
- Chaque entité est stockée avec:
  - Un type
  - Une position dans le texte
  - Un score de confiance (probabilité estimée par le modèle NER que l'entité ait été correctement identifiée)

Relations: Approche 2 niveaux

Niveau 1: Syntaxique (SpaCy)

- Patron : ENT_A —[verbe]→ ENT_B
- Structures prép. : in / of / by
- Définitions : X is a Y
- C_syntax : basé sur la structure grammaticale, le type de dépendance et la distance entre les entités.

Niveau 2: Consolidation LLM

- Le triplet et son contexte sont fournis au LLM via un prompt contraint.
- C_combined = 0.4×C_syntax + 0.6×C_llm
- ≥0.85 → auto | 0.70-0.85 → review | <0.70 → rejet

Étapes 5 & 6: Augmentation LLM & Alignement Ontologique

Étape 5: Augmentation sémantique LLM

- Relations implicites non présentes dans le texte

Ex: Apple — est le fabricant de → iPhone

Apple → développe → iPhone

- Inférence de types conceptuels

Ex: Apple rdf:type Organization

- **Marquage explicite :** Toutes marquées : is_hypothetical = true
  - Validation humaine obligatoire

Étape 6: Alignement ontologique

Mode Découverte (sans ontologie)

- Relations → rdf:Property
- Types générés → owl:Class
- Ontologie légère ad-hoc générée

Mode Alignement (ontologie fournie)

- Entités → classes existantes
- Relations → propriétés ontologiques
- Aide LLM pour choix de propriété
- RDFLib · OWLReady2 · SPARQL

04

Implémentation Technique

Stack technologique · Algorithmes · Formules de scoring · Interface prototype

Stack technologique

Interface UI **Streamlit**	Orchestration visuelle + st.session_state
NLP Core **SpaCy fr_core_news_lg**	Analyse syntaxique, dépendances, POS, lemmes
Moteur IA **Ollama (llama3/phi3)**	Consolidation sémantique des relations · local
Données RDF **RDFLib 6.x**	Triplets RDF (Sujet–Prédicat–Objet)
Runtime **Python 3.10+**	Logique métier, scoring, intégration API

Debug SpaCy : Texte - Lemme - POS - Dépendance - Tête (NER)

COLONNE	DÉFINITION TECHNIQUE	EXEMPLE CONCRET	IMPACT SUR LA DÉTECTION
Texte	Le token (mot ou ponctuation) brut.	*Apple*	C'est l'unité de base.
LEMME	Forme canonique du mot (racine).	*est* → *être*	Permet de regrouper "est", "sont", "être" sous un même concept relationnel.
POS	**Part-Of-Speech** (Nature grammaticale).	`PROPN` (Nom Propre), `VERB`	Le filtre principal. Le système cherche prioritairement des liens entre `PROPN` via des `VERB`.
Dépendance	Relation grammaticale vers le parent.	`nsubj` (Sujet), `obj` (Objet)	**C'est la carte routière.** Indique si le mot est l'acteur (`nsubj`) ou la cible (`obj`) de l'action.
Tête	Le mot "chef" de ce token.	Pour *Apple*, la tête est *lancé*.	Permet de remonter les fils. Si deux entités pointent vers le même verbe (la tête commune), elles sont probablement liées.

Algorithme Shortest Path & formules de scoring (Extraction des relations)

Algorithme — Shortest Path (arbre déps)

1. Le système identifie les tokens qui appartiennent à des entités (ex: Token 1 à 2 = "Apple")

2. Pour chaque paire d'entités, il cherche leur plus proche ancêtre commun dans l'arbre (souvent le verbe principal)

3. Il liste tous les mots significatifs entre les deux entités.:

Formules de Scoring

Score Syntaxique C_syntax

$$C_syntax = \max(0.4 , 1.0 - (D \times 0.02))$$

D = distance en mots · plancher = 0.4 · pénalité = 2%/mot

Score Hybride C_combined

$$C = \alpha \times C_syntax + \beta \times C_llm$$

α=0.4 (syntaxique) · β=0.6 (LLM) C_llm = 0.9 (fixé si réponse cohérente)

Seuils de décision

≥ 0.85 → auto_validated

0.70–0.85 → needs_review

< 0.70 → rejected

Flux de données technique: Modules Python

`nlp_processor.py` *Prétraitement*	Tokenisation · Fallback Noun Chunks si NER vide
`relation_extractor.py` *Extraction*	Arbre syntaxique · Shortest Path · Relations candidates brutes
`llm_service.py` *Consolidation LLM*	API Ollama locale · Prompt “*Normalise la relation 'Apple' 'lancé' 'iPhone' en propriété RDF concise.*” · Normalisation RDF
`app.py` *Validation*	Calcul des scores · Présentation à l’utilisateur · Human-in-the-loop
`rdf_builder.py` *Sérialisation*	Conversion Python → URI RDF · Génération Turtle (.ttl)

Démonstration

Limites & perspectives

Limites du POC

- ✗ Extraction symbolique seule : relations complexes mal détectées
- ✗ LLM : hallucination de relations non présentes dans le texte
- ✗ Scalabilité limitée : conçu pour textes courts (POC)
- ✗ Dépendance au modèle SpaCy : qualité variable selon la langue
- ✗ Alignement ontologique partiel : pas de résolution de conflits

Perspectives d'amélioration

- ✓ Modèles NER fine-tunés sur domaine spécifique
- ✓ Règles d'extraction enrichies + patterns métier
- ✓ Mécanismes de validation formelle de cohérence OWL
- ✓ Extension à des corpus plus larges (scalabilité)
- ✓ Intégration d'ontologies de référence (DBpedia, Wikidata)

Conclusion

1. 4 méthodes analysées et comparées
2. Pipeline hybride en 8 étapes : NLP + LLM + RDF + Human-in-the-loop
3. Prototype fonctionnel : Streamlit · SpaCy · Ollama · RDFLib
4. Scoring de confiance : C_combined = 0.4*C_syntax + 0.6*C_llm
5. Export RDF (Turtle) pour le Web Sémantique

Bibliographie

- Agrawal, G., Deng, Y., Park, J., Liu, H., & Chen, Y.-C. (2023). *Building Knowledge Graphs from Unstructured Texts: Applications and Impact Analyses in Cybersecurity Education.*
- Atzori, M. (2022). *Extracting Data from Text and Querying it via KGQA and BESt Queries.* University of Cagliari.
- Novelli, A. D. P., & Oliveira, J. M. P. (2011). *Simple Method for Ontology Automatic Extraction from Documents.*
- Debatty, T., Michiardi, P., Thonnard, O., & Mees, W. (2014). *Scalable Graph Building from Text Data.* Proceedings of the 3rd International Workshop on Big Data, Streams and Heterogeneous Source Mining (BigMine). *Proceedings of Machine Learning Research*, 36, 120–132.